

Chrome Developer Tools

The Chrome Developer Tools is a lightweight set of tools that is already integrated into the Chrome web browser by Google. This set of tools provides a more in depth view of client side data and issues. While Glimpse and Prefix are focused on the performance of the server and database, Chrome provides detailed analytics about view rendering as well as the impact of content and content type on performance. Dev Tools also allows JavaScript profiling and provides some of the same server side performance data as the other tools. For these reasons, and because it is already pre-installed and available to any developer using the Chrome web browser, the Chrome Developer Tools is included as part of the performance diagnostic design for CW Master Teacher.

Table of Contents

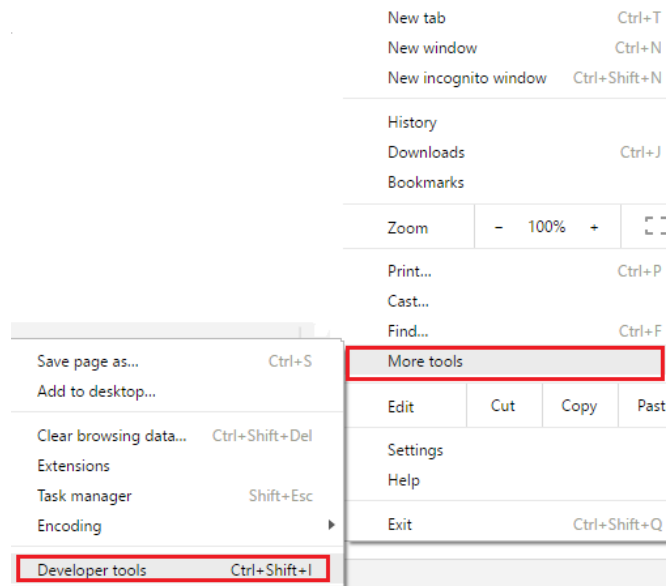
Installation	1
Setup	1
Network	2
Network Panel Overview	2
Discover Details for a Single Resource	3
Disable Cache Button	4
Record Network Activity	4
Capture Network Activity Screenshots	4
View DOMContentLoaded and Load Time	5
Timeline	6
Timeline Panel Overview	7
Record Scripting Activity	8
Record Memory Activity	8
Capture Screenshots	8
Save and Load Recordings	8
Summary Tab	9
Zoom in on a Timeline Section	9
Profiles	9
Record A CPU Profile	10
View CPU Profile	10
View Function Details	10
Investigate Memory Allocation By Function	11

Install Chrome Browser

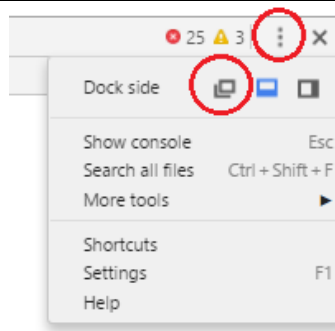
1. Download Chrome from <https://www.google.com/chrome/>
2. Follow Installer instructions

Open DevTools

Select **“Customize and control Google Chrome”** button on the right upper corner of the Chrome window. Then, select **More Tools** in the drop-down list, and click on **Developer tools**, the last one on the drop-down list.



Open the DevTools In Another Window (Recommended)



Network

The network panel helps the developers to understand how each network operates on a page by looking into the requested and downloaded resources.

Goal:

3. Analyze the network activity and load information for each resource on the website.
4. Define which part of the networking process is causing the website to run slow.

Precondition:

Developer Tools experiments needs to be enabled to take screenshots of the website.

1. Go to **chrome://flags** on browser.
2. Enable **Developer Tools experiments**.

Developer Tools experiments Mac, Windows, Linux, Chrome OS

Enables Developer Tools experiments. Use Settings panel in Developer Tools to toggle individual experiments. [#enable-devtools-experiments](#)

Disable

Basic Functionalities Overview:

- Record network performance.
- Explore load information and individual resources.
- Filters and sort the resourced on the **Requests Table**.

Network Panel Overview

1. **Controls.** These options are used to perform the **Network** Panel functions.
2. **Filters.** These options are used to sort the resources that are displayed in the **Requests Table**.
 - Check the type of resources and click the **Filters** button .
3. **Overview.** This graph shows a timeline of when resources were retrieved.
4. **Requests Table.** This table lists out every resource that was retrieved.
 - **Name:** The name of the resources.
 - **Status:** The HTTP status code. (add 404, 200, 300)
 - **Status 304:** The browser has already downloaded the resource before, and it has local or cache copy, so the resource does not have to be downloaded again.
 - **Status 200:** Standard response for successful HTTP requests. (All the resources get re-downloaded after refreshing the page.)
 - **Status 400:** The requested page is not find. (Time to remove the bad request in the codes.)
 - **Type:** The MI ME type of the requested resources.
 - The resources can be JavaScript, CSS, Image, Media, Doc..., etc.
 - **Initiator:** The object or process that initiated the request.
 - **Size:** The size of the resources.
 - Define which file might take longer to deliver by the server.
 - **Time:** The total duration time from the stat of the request to the end of the

response.

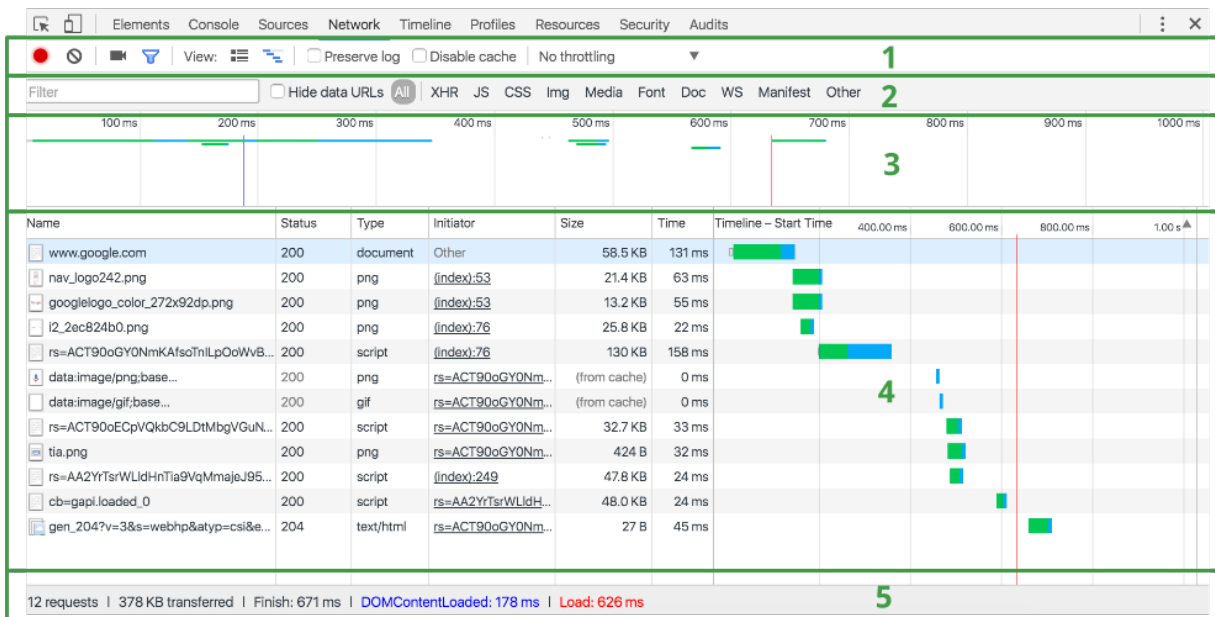
- **Timeline:** A visual waterfall of all network requests.

There are many options to display requests and response times:

- **Start Time:** The start time of each request.
- **Response Time:** Each request's response time.
- **End Time:** The time when each request completed.
- **Duration:** The total time of each request.
- **Latency:** The time between the start of the request and the beginning of the response.

5. **Summary.** This panel shows the exact load time of the website and the initial markup of a page has been parsed.

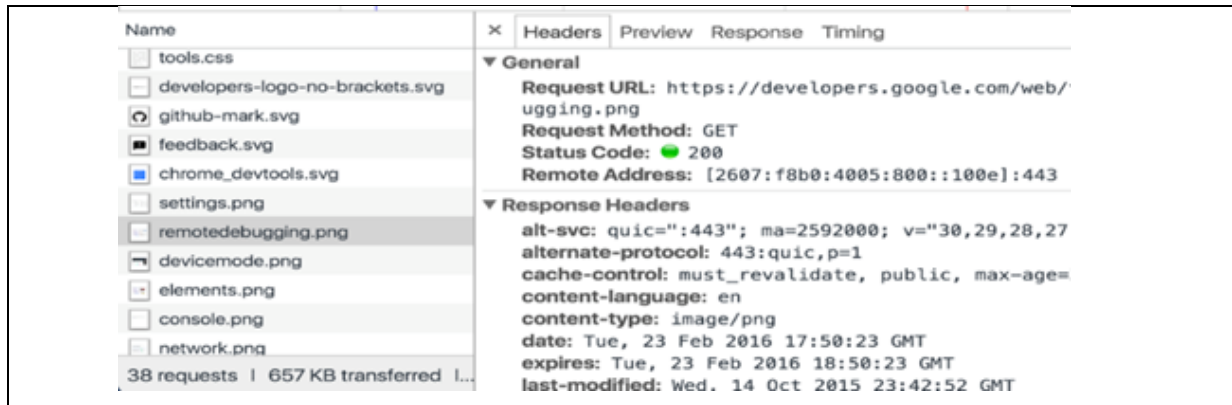
Five Panels of Network Tab:



Discover Details for a Single Resource

Click on a resource name under the **Name** column of the **Requests Table**.

1. **Headers:** Resource's HTTP headers.
2. **Preview:** Previews of JSON, image, and text resources.
3. **Response:** HTTP response data.
4. **Timing:** The lifecycle for the resource of the request.



Disable Cache Button

Check the **Disable Cache** button. This increases the accuracy of analyzing a page. When measuring the exact loading time of the website, the developers do not want to time the website that already has the resources stored in the cache.



Record Network Activity

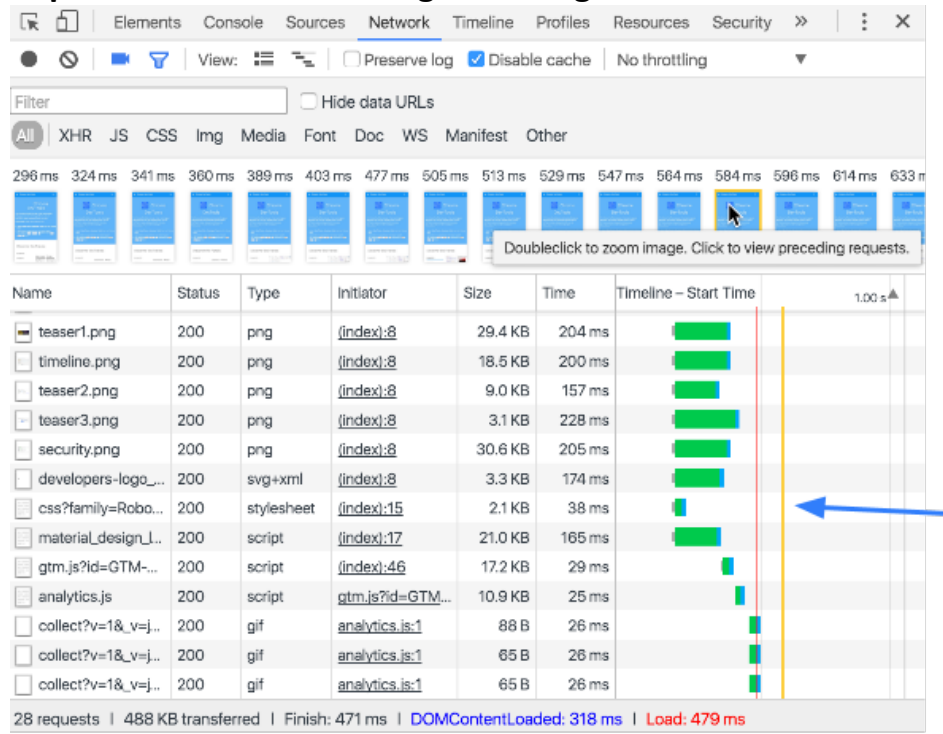
1. Press the red (●) button (The button is grey when DevTools is not recording.)
2. Interact with the page.
3. Press the red (●) button again when it is finished.

Capture Network Activity Screenshots

1. Press the (■) button (The button is grey when DevTools is not recording.)
2. Interact with the page.

3. Press the () button again when it is finished.

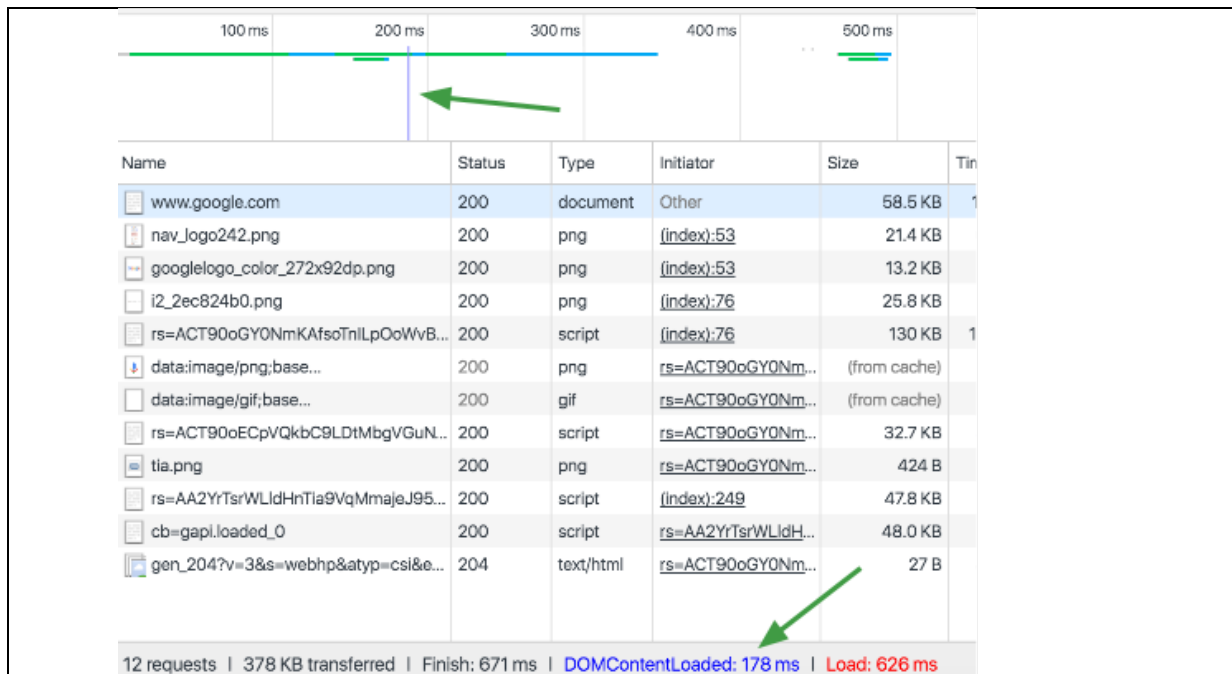
Capture screenshots during recording:



View DOMContentLoaded and Load Time

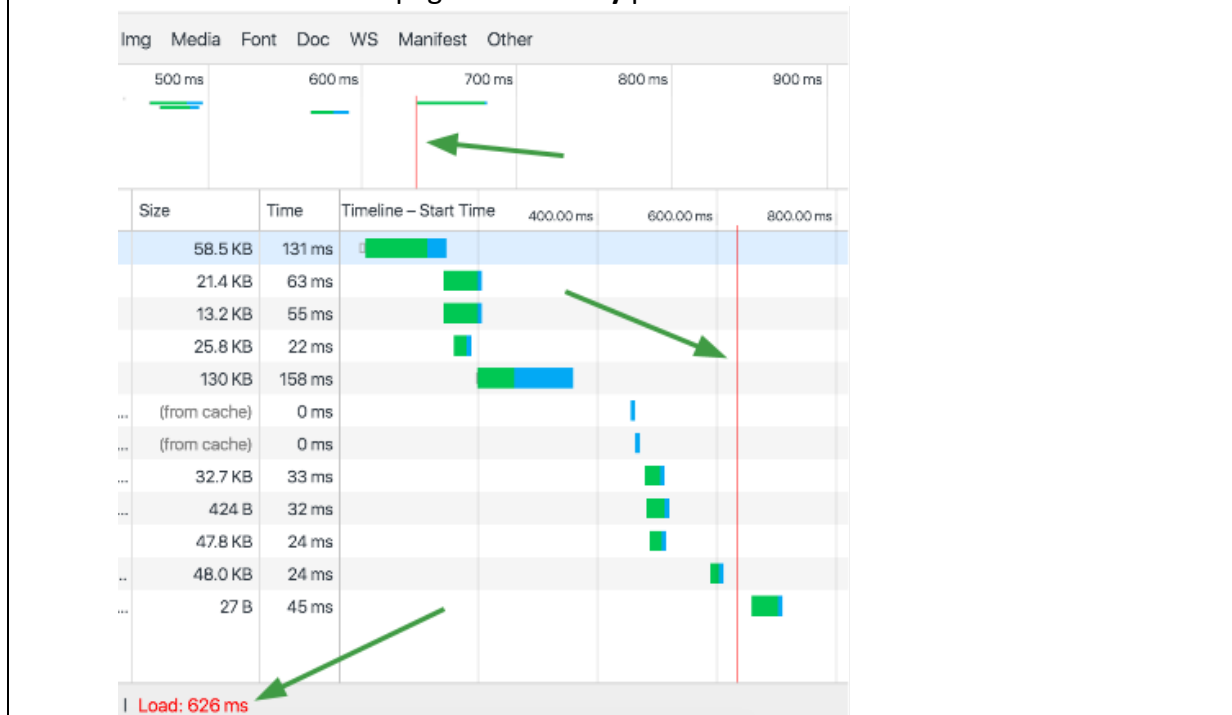
DOMContentLoaded Time: (The initial markup of a page has been parsed).

- The Blue vertical bar in the **Overview** panel.
- At the bottom of the page in **Summary** panel.



Load Time: (The exact time of the event).

- The Red vertical bar in the **Overview** panel.
- At the bottom of the page in **Summary** panel.



Timeline

The timeline panel helps the developers to improve the runtime performance by understanding the execution time and how the events happened during the lifecycle of the website.

Goal:

1. Investigate what sort of functions and scripts that are causing the website to have potential scrolling problems.
2. Explore how the website is running by looking at its constant, frame per second, output screenshot, scripting, and functions.

Precondition:

Developer Tools experiments needs to be enabled to take screenshots of the website.

1. Go to **chrome://flags** on browser.
2. Enable **Developer Tools experiments**.

Developer Tools experiments Mac, Windows, Linux, Chrome OS

Enables Developer Tools experiments. Use Settings panel in Developer Tools to toggle individual experiments. [#enable-devtools-experiments](#)

Disable

Basic Functionalities Overview:

- How to look at the website performance.
- Analyze the runtime performance.

Timeline Panel Overview

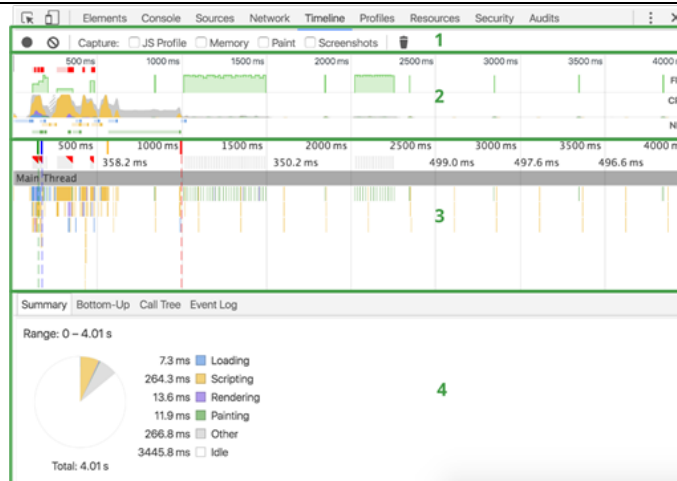
1. **Controls.** These options are used to perform recording functions.
2. **Overview.** This graph shows a high-level summary of a page performance.
 - **FPS:** Frame Per Second. The higher the green bar, the higher the FPS.
 - **CPU:** Shows which type of the events consumed CPU resource.
 - **Net:** The longer the bar, the longer it took to retrieve the resource.

Colored Bar:

- HTML files are **blue**.
- Scripts are **yellow**.
- Stylesheets are **purple**.
- Media files are **green**.
- Miscellaneous resources are **grey**.



3. **Flame Chart.** A visualization of the CPU stack trace.
 - **Red line:** the *DOMContentLoaded* event.
 - **Blue line:** the *Load* event.
4. **Details.** This panel shows more information about the event and selected time frame.



Record Scripting Activity

This visualizes the JavaScript code issues. Red indicator means that part of the code has the loading issues.

1. Click on the **JS Profile** checkbox.
2. Press the **red (●) button** to record.
3. Interact with the page.
4. Press the **red (●) button** to stop recording.

Record Memory Activity

This visualizes the memory leaks.

(Optional: Click  button while recording to force garbage collection.)

1. Click on the **Memory** checkbox.
2. Press the **red (●) button** to record.
3. Interact with the page.
4. Press the **red (●) button** to stop recording.

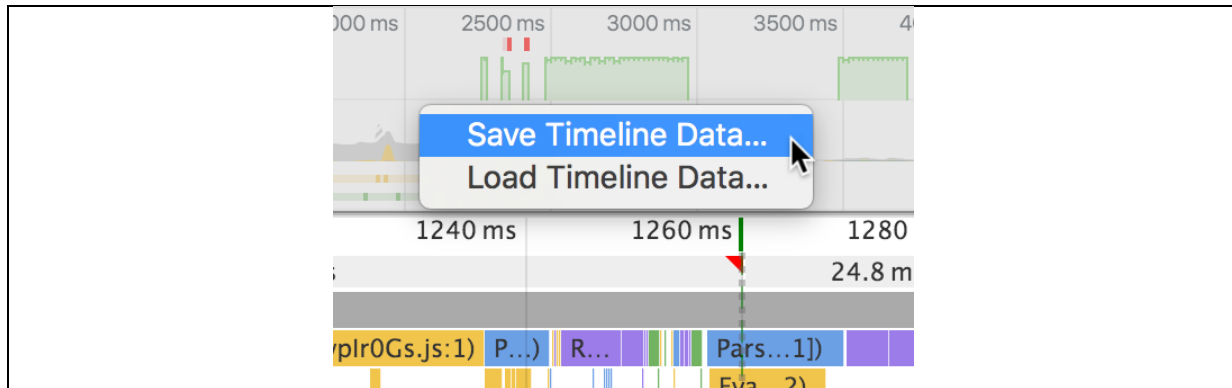
Capture Screenshots

This shows how the website look like in milliseconds.

1. Click on the **Screenshots** checkbox.
2. Press the **red (●) button** to record.
3. Interact with the page.
4. Press the **red (●) button** to stop recording.

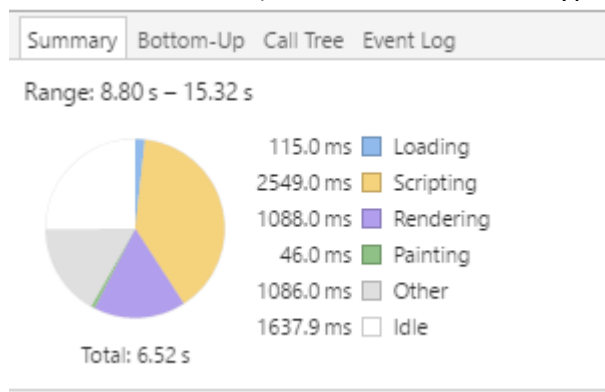
Save and Load Recordings

Save and open recordings by right-clicking inside the **Overview** or **Flame Chart** panels.



Summary Tab

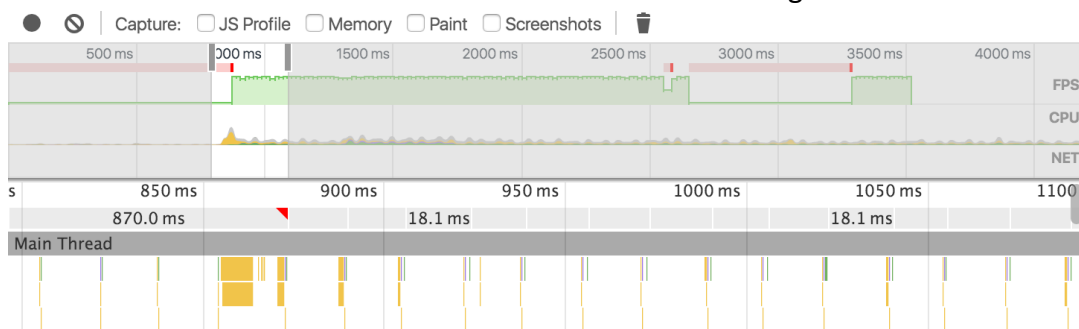
This shows different phases of execution. (Presents for all event type.)



Zoom in on a Timeline Section

Zoom in on a section of a recording to make analysis on the events that happened when running the website.

1. Select out a Timeline section in the **Overview** pane by adjust the gray sliders in the ruler area.
2. W and S for zoom in and zoom out. A and D move left and right.



Profiles

The profiles panel allows developers to collect JavaScript CPU profiles, to take memory heap

snapshots, and to record heap allocations.

Goal:

1. Analyze the execution time for running JavaScript functions.
2. Understand how memory is allocated by JavaScript functions.

Precondition:

- Chrome must be installed and CW Master Teacher website loaded

Basic Functionalities Overview:

- Record exactly which functions were called and how long each took with the CPU Profiler
- Analyze the memory allocated for each JavaScript function

Record A CPU Profile

CPU profiles show where execution time is spent in your page's functions.

1. Select the **Collect JavaScript CPU Profile** radio button.
2. Press **Start**.
3. Depending on what you are trying to analyze, you can either reload the page, interact with the page, or just let the page run.
4. Press the **Stop** button when you are finished.

View CPU Profile

There are three ways to analyze a CPU Profile:

- **Heavy (Bottom Up):** This default view displays which functions had the most impact on performance and examine the calling paths to those functions.
- **Chart:** Shows a chronological flame chart of the recording.
- **Tree (Top Down):** Shows an overall picture of the calling structure, starting at the top of the call stack.

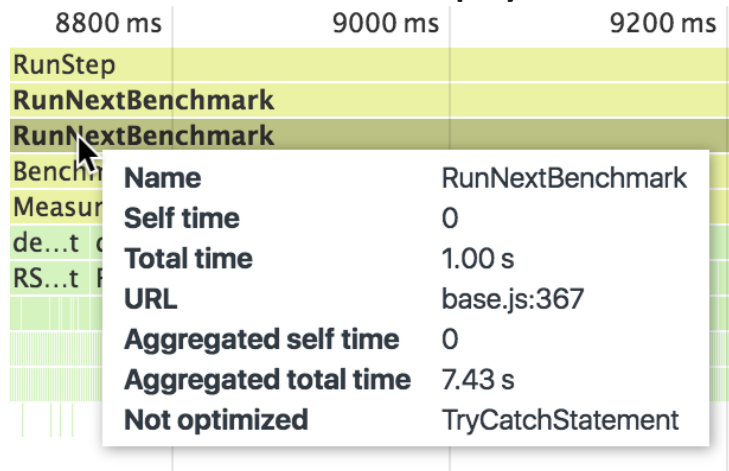
View Function Details

Hover over a function to display its name and timing data. The following information is provided:

- **Name:** The name of the function.
- **Self time:** How long it took to complete the current invocation of the function, including only the statements in the function itself, not including any functions that it called.
- **Total time:** The time it took to complete the current invocation of this function and any functions that it called.
- **URL:** The location of the function definition in the form of file.js:100 where file.js is the name of the file where the function is defined and 100 is the line number of the definition.
- **Aggregated self time:** Aggregate time for all invocations of the function across the recording, not including functions called by this function.
- **Aggregated total time:** Aggregate total time for all invocations of the function, including functions called by this function.

- **Not optimized:** If the profiler has detected a potential optimization for the function it lists it here.

Hover Over a Function to Display Its Name and Timing Data:



Investigate Memory Allocation By Function

View memory allocation by JavaScript function.

1. Select the **Record Allocation Profiler** radio button.
2. Press **Start**.
3. Perform the actions on the page which you want to investigate.
4. Press the **Stop** button when you have finished all of your actions.

Breakdown of Memory Allocation by Function:

Heavy (Bottom Up)					
Profiles	Self Size (bytes)		Total Size (bytes)		Function
	84 448	9.66 %	380 024	43.49 %	▶_render polymer.html:7
	65 680	7.52 %	114 944	13.16 %	▶_parseChildNodesAnnotations polymer.html:5
	49 216	5.63 %	49 216	5.63 %	▶add polymer.html:5
	49 216	5.63 %	49 216	5.63 %	▶store polymer.html:6
	32 896	3.76 %	32 896	3.76 %	▶_unescapeText shop-detail.html:70
	32 848	3.76 %	32 848	3.76 %	▶_annotatedComputationEffect polymer.html:5
	17 512	2.00 %	17 512	2.00 %	▶_configureProperties polymer.html:5
	16 704	1.91 %	16 704	1.91 %	(anonymous) shop-list-item.html:16
	16 512	1.89 %	16 512	1.89 %	▶DoJoin native array.js:97
	16 456	1.88 %	16 456	1.88 %	▶_parseBindings polymer.html:5
	16 448	1.88 %	16 448	1.88 %	▶_parseCss polymer.html:6
	16 448	1.88 %	16 448	1.88 %	▶_createEventHandler polymer.html:5
	16 448	1.88 %	295 576	33.83 %	▶_insertInstance polymer.html:7

Work Cited:

Google Developers. N.p., n.d. Web. 27 Oct. 2016.

<<https://developers.google.com/web/tools/chrome-devtools/>>.